
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 1: Metamodelling
Project Type: Code Generation
September 25, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Purpose of the Project	1
1.1	Motivation and Goals	1
1.2	Refinement of Task 0	1
1.3	Assumptions and Conditions	1
1.4	Scope	1
1.5	Domain Analysis	2
1.5.1	Observations From the Existing Configuration	2
1.5.2	Structural and Operational Concepts	2
2	Alternatives and Justification	2
2.1	Alternatives Considered	2
2.1.1	A Third Metamodel for the Generated Files	2
2.1.2	Grammar First or Metamodel First	3
2.1.3	One Resolved Model, or One Per Application	3
2.2	Chosen Modelling Solution	3
2.2.1	Three Model Stages, Two Metamodels	3
2.2.2	What Each Stage May Contain	3
2.2.3	Shared Values Are Set by the Platform	4
2.2.4	Derived Values Cannot Be Overridden	4
2.2.5	Requirements Instead of Target Fields	4
2.2.6	Enumerations Instead of Free-Form Strings	4
2.2.7	Release Grouping and Deployment Order	4
3	Solution Design	4
3.1	The Source Metamodel	5
3.1.1	The Project Document	5
3.1.2	The Platform Document	7
3.1.3	Enumerations	9
3.2	The Target Metamodel	9
3.2.1	Required Content	9
3.2.2	Structure	9
3.2.3	The Path Plan	11
3.2.4	The Generated Resources	11
3.2.5	Keeping Target Field Names Out of the Model	11
3.3	Concrete Syntax and Constraints	11
3.3.1	The Concrete Syntax	11
3.3.2	The Constraints	11
3.4	Implementation in EMF	12
4	Tests and Evaluation	12

4.1	Testing Strategy	12
4.2	Example Models	13
4.2.1	The Minimal Model	13
4.2.2	A Resolved Model as a Deployment View	13
4.2.3	Example Target Models	13
4.3	Rejected Models	13
4.4	Evaluation Against the Purpose	14
4.5	Limitations	14
4.6	Reflection	14
A	Worked Examples	16
A.1	The Minimal Model, End to End	16
A.2	Continuous and Interrupted Applications in One Project	16
A.3	A Rejected Model	16
A.4	The Hand-in Archive	16
B	All Constraints	18

1 Purpose of the Project

1.1 Motivation and Goals

Today each application's repository holds its own hand-written Kubernetes files. Writing them by hand is hard because services are related: an application's address, its ports and the services it depends on appear in the files of several applications, and a change to one service means edits in the others. Spread over many repositories, these copies drift apart. Gathering all files in one central repository removes the copies, but separates each deployment from the application it belongs to and from the people who maintain it.

The project takes a third route. Each application keeps a short deployment document in its own repository, written in a domain-specific language. The document exposes only what the application should care about: how it must be deployed. It states requirements that hold on any machine, such as how much memory and CPU a process needs and whether it needs a graphics card, instead of naming machines. It also states how the application is reached, by hostname and path, next to the application instead of in a separate router repository. The parts all applications share, such as the cluster's policies and entry points, are written once, in a platform document.

The language makes deployments specifiable and configurable while it enforces the cluster's standards. Deployment behaviour that every application needs, such as blue/green switching, monitoring and backups, is derived once and reused, instead of copied into each application's files. A model transformation turns the documents into a resolved model, and code generation turns the resolved model into the Kubernetes files.

Task 1 defines the two metamodels this needs, in Ecore. The *source metamodel* is the language an author writes. The *target metamodel* describes the *resolved model*, which records every value the platform derives. OCL constraints reject invalid source models, an Xtext grammar gives the source language a textual syntax, and example models show that both metamodels work. Task 2 will implement the transformation with QVT-Operational, and Task 3 the code generation with Acceleo.

1.2 Refinement of Task 0

Task 1 refines the Task 0 proposal in three ways.

- Task 0 described the target metamodel as typed Kubernetes resources. The target metamodel now records the platform's decisions in the terms of the source language. Typed resource classes state only which file a generator writes at which path, and the files' own fields are not modelled (Sections 2.2.5 and 2.1.1).
- Task 0 treated the platform document as an input beside the model. It is now part of the source metamodel, with its own root class (Section 3.1), and constraints check a project document against it (Section 3.3).
- Task 0 listed ten source classes. The source metamodel adds, among others, a migration per application, named ports, alerting, and declarations shared by all processes of a project or application.

1.3 Assumptions and Conditions

- One cluster. The concepts and examples come from its existing, hand-maintained configuration (Section 1.5).
- Application owners write project documents. The platform owner writes one platform document per cluster.
- The platform sets every value that must be unique in the cluster or uses a shared, limited resource (Section 2.2.3).
- The generated files are not applied to a cluster. Correctness concerns the resolved model and the generated files.
- The tools are Eclipse Modeling Tools with EMF, OCL and Xtext, built with Maven and Tycho (Section 3.4).

1.4 Scope

Table 1 sets the scope.

In scope	Out of scope
Two metamodels: the authored source language, and the resolved model.	A metamodel of the generated files. They are text, and are compared as text.
A closed set of six generators for one cluster. Adding a generator requires a recorded decision.	Additional target platforms, and a second cluster.
Validation of a source model, and the constraints that reject an invalid one.	Applying the generated files to a cluster.
Dependency resolution between applications.	Runtime behaviour, scaling decisions and monitoring rules.

Table 1: Scope of the Project

1.5 Domain Analysis

The existing configuration covers one Kubernetes cluster, seven nodes, ten namespaces, about thirty processes and twenty applications. It is maintained by hand.

1.5.1 Observations From the Existing Configuration

Repeated values. One hostname appears seven times across three repositories. Two tests check those copies for disagreement. A port appears in the container specification, service, probe and monitoring rule.

Repeated derivations. Four deployments contain the same rollout block. Comments explain two choices. Recreate stops the old pod before starting the new one, leaving an interval without a running pod. A JVM needs more startup time than a static bundle. A *derivation* computes these values from the authored model and cluster policy.

Mechanisms instead of requirements. The files specify rollout strategies and node labels. They do not state whether an upgrade may interrupt service or what a process needs from a machine. A volume that cannot attach to two nodes at once requires recreate. A rolling replacement can fail if the new pod starts on another node before the old one stops.

Shared values. Hostnames must be unique. Only one process can use a graphics card at a time. The *edge* consists of shared entry points for external traffic (Section 2.2.3).

1.5.2 Structural and Operational Concepts

Structurally, a project groups applications. An application groups processes released together. Processes offer and use named ports. Hostname paths lead to an application’s processes.

Operational concepts specify machine requirements, data durability, secret access and delivery, health checks, startup time and whether versions may run together during an upgrade.

2 Alternatives and Justification

The starting point is how the cluster is configured today: Kubernetes files written by hand in each application’s repository. The related values between services make these files hard to keep consistent, and each application repeats the same deployment behaviour. The other alternatives either keep these problems or move them elsewhere. This section first compares them (Section 2.1), and then gives the decisions of the chosen solution in general terms (Section 2.2). Section 3 describes the resulting classes in detail.

2.1 Alternatives Considered

Table 2 lists the rejected alternatives, starting with the ways of writing the Kubernetes files directly.

2.1.1 A Third Metamodel for the Generated Files

A third metamodel could support structural output checks, but a maintained copy of the target schema could drift. It adds no constraint beyond the resolved metamodel. The resolved metamodel contains typed resource

Alternative	Why it was rejected
Hand-written Kubernetes files in each application's repository	Related values, such as addresses and ports, are copied between repositories and drift apart. Every application repeats the same deployment behaviour.
All Kubernetes files in one central repository	Removes the copies, but separates each deployment from the application it belongs to and from the people who maintain it.
Template the target files directly (Helm, Kustomize overlays)	No abstract model supports constraints or shared derivations. Repeated values remain.
Annotate the target files with extra fields	The existing configuration gave authored and resolved forms the same name. Errors could not identify which form was wrong.
Two stages, with the generator resolving values internally	Generator decisions appear in no reviewable model.
A third metamodel for the generated files	It would restate the target schema.
A grammar first, with the metamodel inferred	Two syntaxes need the same abstract concepts.
Free-form strings instead of enumerations	The existing free-form field was wrong in every application.
An override mechanism for derived values	Per-process exceptions would conceal an incorrect general rule.

Table 2: Alternatives Considered and Why Each Was Rejected

classes (Section 3.2.4).

2.1.2 Grammar First or Metamodel First

Source models have two serialisations: YAML for authors and XMI for Eclipse. If the YAML grammar inferred the metamodel, it would determine the concepts in both forms. Defining Ecore first gives both serialisations the same classes.

2.1.3 One Resolved Model, or One Per Application

The resolved model covers all applications. Edge routing, deployment order and reverse dependencies require a cluster-wide view. Reverse dependencies identify which applications depend on a given application. Separate per-application computations could disagree or omit values.

Each repository receives a *projection*: its application's part of the resolved model, selected by filtering.

2.2 Chosen Modelling Solution

Seven decisions shape both metamodels. Each follows from the problems of the alternatives above.

2.2.1 Three Model Stages, Two Metamodels

Task 2 transforms an authored model into a resolved model. Task 3 generates text from the resolved model. Only the authored and resolved models have metamodels (Section 2.1.1).

These stages are not metalevels. No `instanceOf` relation connects them.

2.2.2 What Each Stage May Contain

A *decision* is a platform-computed value, such as eligible machines. **Authors state requirements, not target mechanisms (Section 1.5.1). Generators use the resolved model without computing decisions.** The resolved model records computed values.

An earlier two-stage design let the generator compute values internally. It produced three incompatible documents with one version identifier. The resolved model records computed values separately from the authored

document.

2.2.3 Shared Values Are Set by the Platform

The platform sets cluster-unique values and allocations of shared resources. Applications declare other values, which the transformation copies unchanged.

Applications state memory and CPU requirements so processes reserve capacity. The platform compares each requirement with individual machines. It does not sum reservations across the cluster, so it cannot detect aggregate overcommit.

2.2.4 Derived Values Cannot Be Overridden

The platform derives values from the authored model and platform document. There is no override mechanism (Table 2).

One process starts faster and had a shorter hand-set deadline. A rule can derive that deadline from its declared startup budget without an override.

2.2.5 Requirements Instead of Target Fields

Attributes describe requirements, such as writable paths, instead of target fields such as a security context (Section 1.5.1). Only generators contain target field names.

2.2.6 Enumerations Instead of Free-Form Strings

Fixed sets of values use enumerations. Constraints can cover their literals. Free-form strings have no finite set to check (Table 2).

2.2.7 Release Grouping and Deployment Order

An application groups processes released together. The dependency graph determines application order. Using that graph for release grouping joined all applications into one unit through transitive dependencies.

3 Solution Design

The solution consists of the source metamodel, the target metamodel, the concrete syntax and OCL constraints of the source language, and their implementation in EMF. Table 3 says where each is in the hand-in archive (Appendix A.4).

Artefact	In the hand-in	Section
Source metamodel	<code>task-1-metamodelling/model/project-intent.ecore</code>	3.1
Target metamodel	<code>task-1-metamodelling/ model/resolved-deployment.ecore</code>	3.2
OCL constraints	<code>task-1-metamodelling/model/project-intent.ocl</code>	3.3
Concrete syntax	<code>implementation/emf/bundles/syntax</code> , two Xtext grammars	3.3
Implementation and tests	<code>implementation/emf</code>	3.4, 4

Table 3: The Artefacts of Task 1

Figures 1 to 3 are UML class diagrams. Ecore implements the EMOF constructs of classes, attributes, references and multiplicities, and a class diagram shows them directly. A deployment view shows a model instance instead, as in Figure 4.

3.1 The Source Metamodel

The source metamodel is `model/project-intent.ecore` in the archive. A *project* document specifies application requirements. One *platform* document specifies cluster facts and policies (Section 2.2.3).

3.1.1 The Project Document

An application using a database declares a migration. One application migrates each project's database. **Migration** has three subclasses. The platform applies a **ManagedMigration** changelog before the new version starts. **SelfMigration** leaves schema changes to the image. **NoMigration** leaves them to another application. **cutover** specifies whether service continues during an upgrade. For **continuous**, the new version receives traffic after the application's processes pass metric checks. This is a *blue/green* switch. For **interrupted**, the old version stops first. **lifecycle** is **application**, **job** or **prepare**. A *prepare process* performs one-off setup after migration and before the new version starts. It serves no traffic.

An **Exposure** belongs to an application because its hostname can route different paths to different processes. A **Surface** is a named port a process offers. A route refers to a surface of its own application. A dependency refers to a surface of another application.

Class	What it represents
Project	The authored document: an owner and the applications it groups.
Application	A unit released together, with a cluster-unique identifier.
Process	One containerised process: image, kind, ports, needs and health.
Surface	A named port a process offers, declared once.
Exposure	A hostname the application is reachable at, and who may reach it.
Route	A path on that hostname, and the process and surface that serve it.
Placement	What a process needs from a machine, as hard requirements.
Volume	Process storage and its durability class, which sets how loss is handled.
Probe	The endpoint that serves a process's readiness or liveness check.
DependencyEdge	A link to another application's named surface. Where the provider has databases, it states how the derived database credential may rotate.
Credentials	The rotation an application supports for the database credential derived for its dependency.
Migration	How the schema is migrated: by a Liquibase changelog the platform applies, by the image itself, or not at all.
Grant	A secret a process may read, how it is delivered, and its rotation.
Asset	A configuration file required by a process.
Observability	Where metrics are collected, and the alert level for failures.
Sidcar	A helper process sharing the main process's lifetime.
Capacity	An instance count above one, with the reason it is not derived.

Table 4: Classes of the Project Document

Figure 1 shows the classes in Table 4 and their relationships.

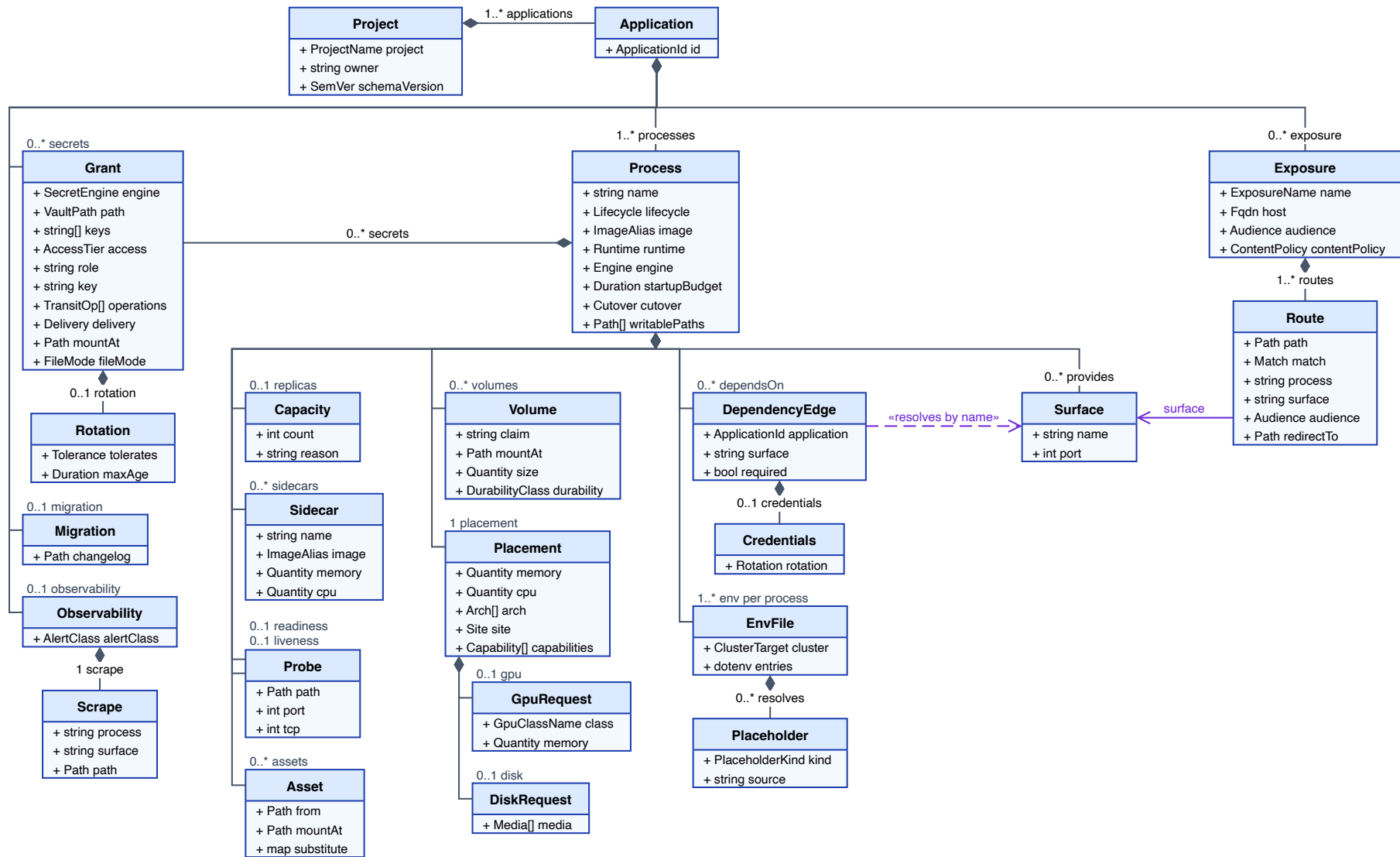


Figure 1: The Project Document

3.1.2 The Platform Document

The platform document specifies shared cluster facts and policies. For example, it states whether secrets are encrypted at rest (Section 2.2.5). A tier refers to the application that runs its proxy. The delivery policy refers to the *delivery machinery*: applications that switch other applications to a new version. Constraints reject secret delivery without encryption and storage requests without matching policies (Section 3.3).

Class	What it represents
Platform	One document per cluster, with its owner and container security restrictions.
PlatformMetadata	Which cluster and project the document describes, and the digest of the machine contract (the facts each machine publishes) it was written against.
Substrate	Cluster facts that other decisions depend on: its version, datastore, server count, whether secrets are encrypted at rest, and its network plugin and policy controller.
Bootstrap	What must exist before the first generated object can be applied. The document records it, and the generators do not create it.
FluxSource	The source reference of Flux, the tool that fetches the generated files and applies them to the cluster.
RenderedArtifacts	The registry repository each project's generated output is published to, one signed artifact per project.
ArtifactSigner	The keyless identity, an issuer and a subject, whose signature Flux accepts on a published artifact.
VaultState	Whether the secret store is unsealed, the second bootstrap fact.
Tier	An entry point of the edge: its audiences, listener, certificate source, authentication endpoint, and the declared Application that acts as its proxy.
DurabilityPolicies	The three durability classes' policies, held together so a missing one is visible.
DurabilityPolicy	One class's backup window and retention.
OffClusterCopy	Where an irreplaceable volume's copy goes, and the credential that writes it.
EnginePolicies	The backup method per engine, held together for the same reason.
EnginePolicy	One engine's backup method: the alias of an image that performs the backup. The document contains no command or script.
MonitorCadence	How often metrics are collected, and the collection timeout.
ProbeCadence	The cluster's probe timing: period, timeout and the number of failures tolerated.
EphemeralPolicy	The size a declared writable path receives.
Provider	A service the cluster runs that this model does not deploy, with its address and named surfaces, which a dependency may refer to.
MigrationPolicy	The runner image alias, deadline and resources for managed migrations.
DeliveryPolicy	The delivery machinery: the applications that switch other applications to a new version and are not switched themselves.
AnalysisPolicy	How often a new version is checked, how many passing checks promote it, and how many failing ones roll it back.
HandoverLedger	Projects on the old path and projects delivered from pinned artifacts. It also records the old path's removal date.

Table 5: Classes of the Platform Document

Figure 2 shows the classes in Table 5 and their relationships.

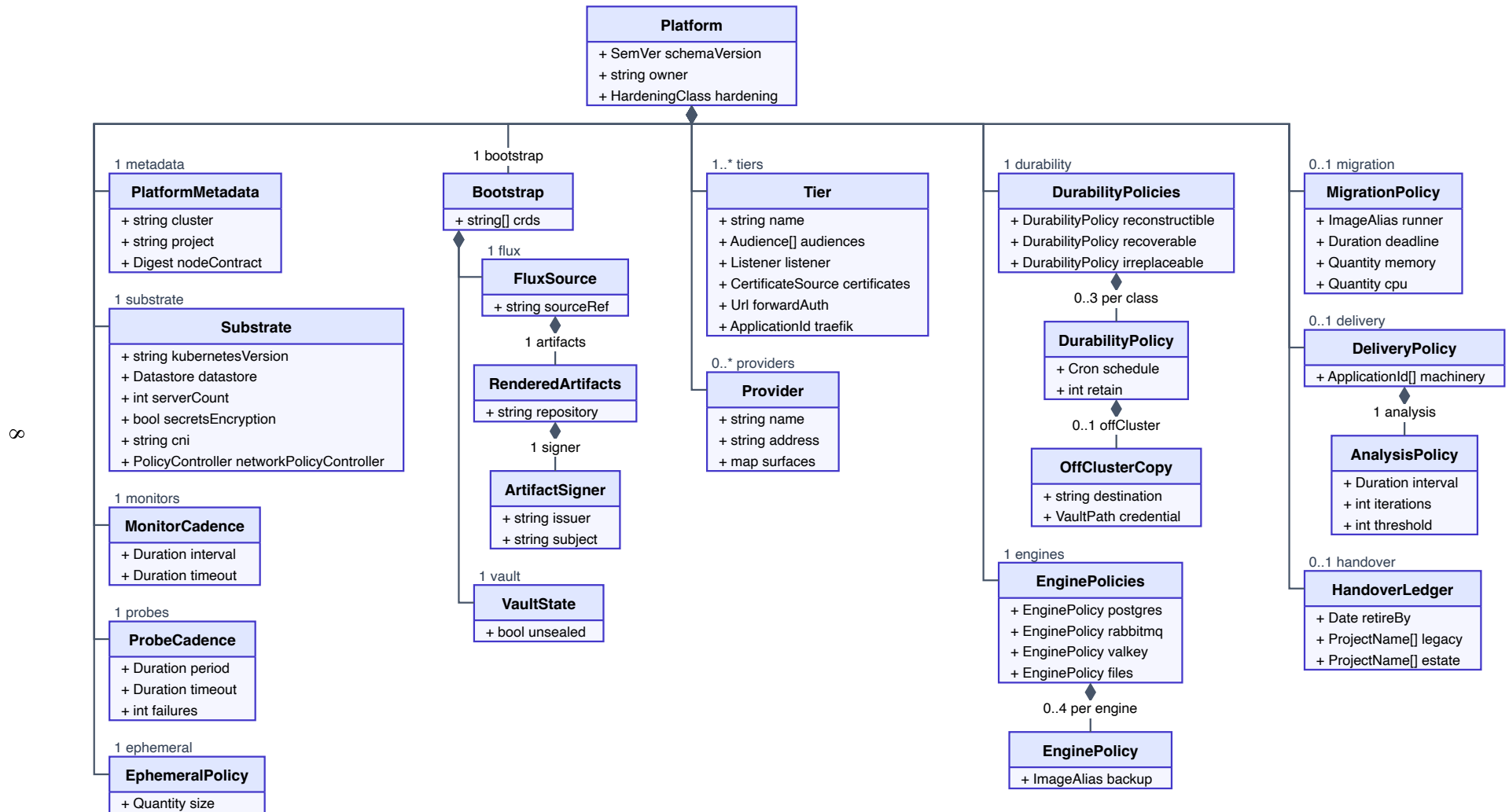


Figure 2: The Platform Document

3.1.3 Enumerations

Figures 1 and 2 show fixed value sets as types such as `Cutover`, `DurabilityClass` and `Listener` (Section 2.2.6).

3.2 The Target Metamodel

The target metamodel is `model/resolved-deployment.ecore` in the archive. Task 2 produces its instances. Task 3 generates files from them. A resolved model records platform decisions, typed resources and output paths.

3.2.1 Required Content

Generators use the resolved model (Section 2.2.2).

Decisions. Derived values include edge tiers, eligible machines, process identity, image digests, probe timings, backup plans and network peers. *Switchover* specifies how a new process version replaces the old: **blue-green**, **stop-start** or, for delivery machinery, **rolling**.

Order and atomicity. *Reconcile units* group objects Flux applies together. Dependency edges order these groups. For a blue/green application, the *release gate* controls when the new version receives traffic. Its inputs include members, checks, an interval and a deadline.

Release identity. An application's *revision* hashes its resolved element, excluding the revision field. Migrations and release gates use that revision to identify the same release.

Provenance. A run hash and input digests identify project documents, platform policy, the machine contract, pinned image digests and observed cluster state. Identical inputs must produce identical output.

3.2.2 Structure

Figure 3 shows the metamodel. The root class, `ResolvedDeployment`, covers all applications (Section 2.1.3). It contains the provenance, the resolved applications, the reconcile units and the path plan.

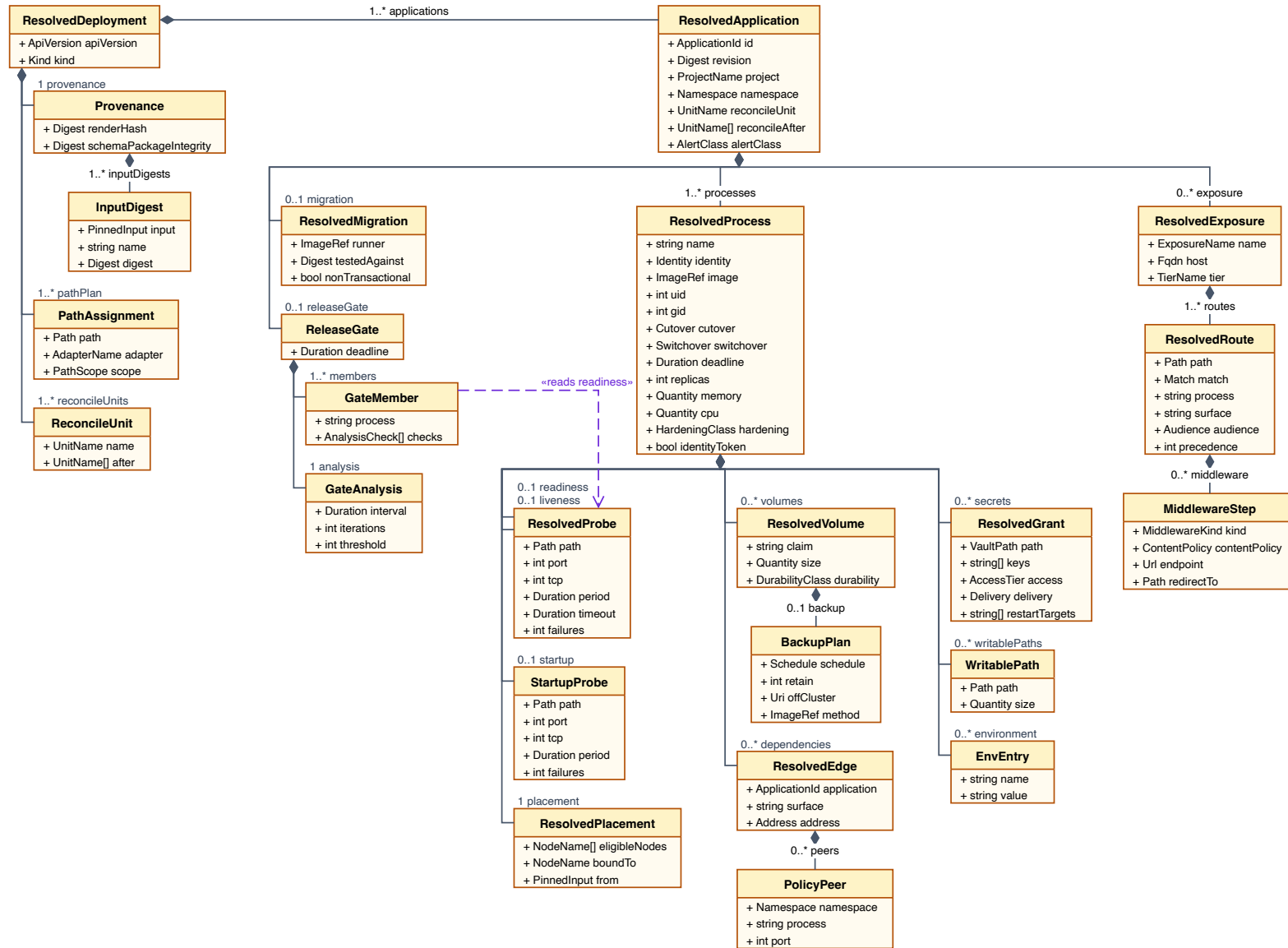


Figure 3: The Target Metamodel

A `ResolvedApplication` also forms the projection published to its repository. Only that published projection has separate provenance.

`ReleaseGate` is absent when no process switches blue/green. An interrupted application stops before the new version starts. The delivery machinery updates in place. `ResolvedMigration` records changesets that cannot run in a transaction and cannot be undone automatically. Migration identity, owner role and deadline follow from the application and platform document.

The `cutover` and `switchover` of a `ResolvedProcess` are unset for a prepare process, which replaces no running version.

3.2.3 The Path Plan

The *path plan* assigns each generated object to a generator and output path (Section 2.2.2). Earlier generator-chosen paths duplicated a group object across applications and placed a cluster object in an arbitrary namespace. The path plan lets the platform detect output-path collisions before generation.

3.2.4 The Generated Resources

Six generators produce workloads, network policy, monitoring, edge routing, secret-store policies and secret delivery (Table 1). A *Canary* lets Flagger switch traffic to a new version.

Typed resources. The output consists of Kubernetes, Flux, Flagger, Traefik and Vault files. Decision classes specify their content. Typed resource classes specify output paths and generators. They include `WorkloadFile`, `SwitchoverFile`, `NetworkPolicyFile`, `MonitorFile` and `IngressRouteFile`. The remaining types are `SecretFile`, `IdentityFile`, `NamespaceFile` and `IndexFile`. A model-to-text template writes each typed resource.

3.2.5 Keeping Target Field Names Out of the Model

`SwitchoverFile` records a blue/green switch (Section 2.2.5). Its generator writes a *Canary*. The model also records hardening, writable paths and middleware steps instead of target syntax.

3.3 Concrete Syntax and Constraints

The source language has a textual syntax and OCL constraints that reject invalid models.

3.3.1 The Concrete Syntax

Authors use the YAML subset found in the existing configuration. Two Xtext grammars define this syntax: one for `platform.intent.yml` and one for `*.project.yml` (Section 2.1.2). Both use source metamodel classes. YAML keys may appear in any order.

A route refers to a declared process and named port. An undeclared process produces a reference error, `E_UNKNOWN_PROCESS`. Handover project identifiers remain strings checked by an invariant because projects are separate documents.

3.3.2 The Constraints

Complete OCL defines 36 invariants and 9 `def` helpers for the source metamodel. The invariants use 28 distinct codes. Three occur on multiple context classes.

Each invariant has its diagnostic code as its name and the rejected object's class as its context. Its diagnostic identifies the rule and the object to fix. Table 6 samples the scopes. Appendix B lists all 28 codes.

Ten project-context constraints use another document: the platform document or a database provider's project file. The invariant's context identifies the object to fix. `allInstances()` searches the loaded resource set. Guards skip cross-document checks when a project file is loaded alone (Example 1).

Code	Context	What it rejects
E_CUTOVER_UNHONOURABLE	Process	One object: a continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	One document: an application whose processes have different cutover values, although it switches as one unit.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	The platform document: a tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	Across documents: an audience no edge tier in the platform document serves.

Table 6: One Constraint for Each Scope

```
context Exposure
  inv E_NO_TIER_FOR_AUDIENCE:
    Tier.allInstances()->notEmpty() implies
      Tier.allInstances()->exists(audiences->includes(self.audience))
```

Example 1: E_NO_TIER_FOR_AUDIENCE, with a guard for a project file loaded alone

EMF reads an unset enumeration as its first literal. Optional enumerations use an unwritable **absent** literal, which invariants test instead of **null**. Constraints over every project document at once, such as unique application identifiers, are not part of the OCL constraints.

3.4 Implementation in EMF

The artefacts are Eclipse plug-in projects in `implementation/emf/`, built by Maven and Tycho on JDK 21. Three projects make up Task 1.

- **metamodel** holds the two `.ecore` files, their `.genmodel` files and the Complete OCL file. The build generates the metamodels' Java classes from the genmodels.
- **syntax** holds the two Xtext grammars and the generated editor. The editor marks an OCL violation on the line of the authored file that causes it.
- **cli** holds a program that reads a set of authored files and writes either the parsed model or the *diagnostics* that reject it. A diagnostic holds the code, the file and the path of the rejected object in that file.

The program checks in three steps and stops after the first step that finds an error. It parses each file, validates each parsed model against the OCL constraints, and then validates the files of a set together, for the constraints across documents. It also writes each parsed model as XML, named after its file: `notes.project.yml` becomes `notes.project.xmi`.

`./mvnw verify` in `implementation/emf` builds the projects and runs the tests of Section 4. The projects `resolve` and `render` contain the start of Tasks 2 and 3.

The hand-in's `task-1-metamodelling` project holds the metamodels and every model as XML, so the models can be inspected without a build (Appendix A.4). Its copy of `project-intent.ecore` embeds the OCL invariants, so Eclipse's Validate reports them directly.

4 Tests and Evaluation

4.1 Testing Strategy

The metamodels are tested by instantiating them. Accepted models show that the metamodels represent the intended deployments, and rejected models show that the constraints reject invalid ones.

- **Accepted models.** Five project documents and one platform document instantiate the source metamodel (Section 4.2). They are modelled on applications of the existing configuration. One hand-written resolved model instantiates the target metamodel.
- **Rejected models.** Thirty-three models each break one rule (Section 4.3).

- **Automated checks.** Every build parses each accepted and rejected model with the Xtext grammars, validates it against the OCL constraints, and compares the outcome with a committed expected result: the parsed model for an accepted model, and the codes and object paths for a rejected one (Section 3.4). A change to a metamodel, grammar or constraint that alters any outcome fails the build. Further tests check the hand-written resolved model against the target metamodel: it validates, a prepare process has no **cutover**, only a blue/green application has a release gate, and the path plan assigns a path to every file of the expected output.
- **Manual checks.** The hand-in holds each model as YAML and XMI. Validating an XMI in Eclipse repeats the check (Appendix A.4).

4.2 Example Models

Table 7 lists five accepted source models. Appendix A.4 gives their paths.

Example	What it tests
minimal	The smallest complete model: one project, one application, one process, and only required values.
auth	Two continuous processes in one application, so the release gate has two members and its deadline is the maximum over the members. It also has a managed migration, because the application uses a database.
data	Several applications in one project share a namespace. Each volume's durability class determines its backup plan.
knowledge	Two applications with different cutover values: a continuous API that migrates the project database with a changelog, and an interrupted worker with a volume. The worker has no migration. The example also has dependencies, shared secrets and routes with different audiences.
delivery	Two applications of the delivery machinery: Flagger and the release gate. Their processes are continuous but are updated in place instead of switched blue/green, because the release gate cannot control its own release.

Table 7: The Example Models and What Each Tests

Appendix A describes two accepted examples and one rejected case in more detail.

4.2.1 The Minimal Model

The minimal model omits rollout strategy, resource limits, security context, labels, selector, namespace, probe timings and instance count. Platform rules derive these values or reject omissions that cannot be derived.

4.2.2 A Resolved Model as a Deployment View

Figure 4 shows the **knowledge** resolved model as a deployment view (Section 3). It shows two processes on their machines, a volume, the edge tier, a hostname and two platform services.

4.2.3 Example Target Models

The hand-written `minimal.resolveddeployment` is the expected Task 2 output and an input for testing Task 3 templates.

4.3 Rejected Models

The archive has 33 rejected models: 18 single project files and 15 pairs of a platform document and a project file. Together they cover all 28 constraint codes and the four codes of unresolved names. Each file of a pair is valid on its own. Validating a rejected model reports the violated invariant, whose name is the code (Appendix A.3). The archive's `refusals/README.md` says which file holds each error, and Appendix B says what each code rejects.

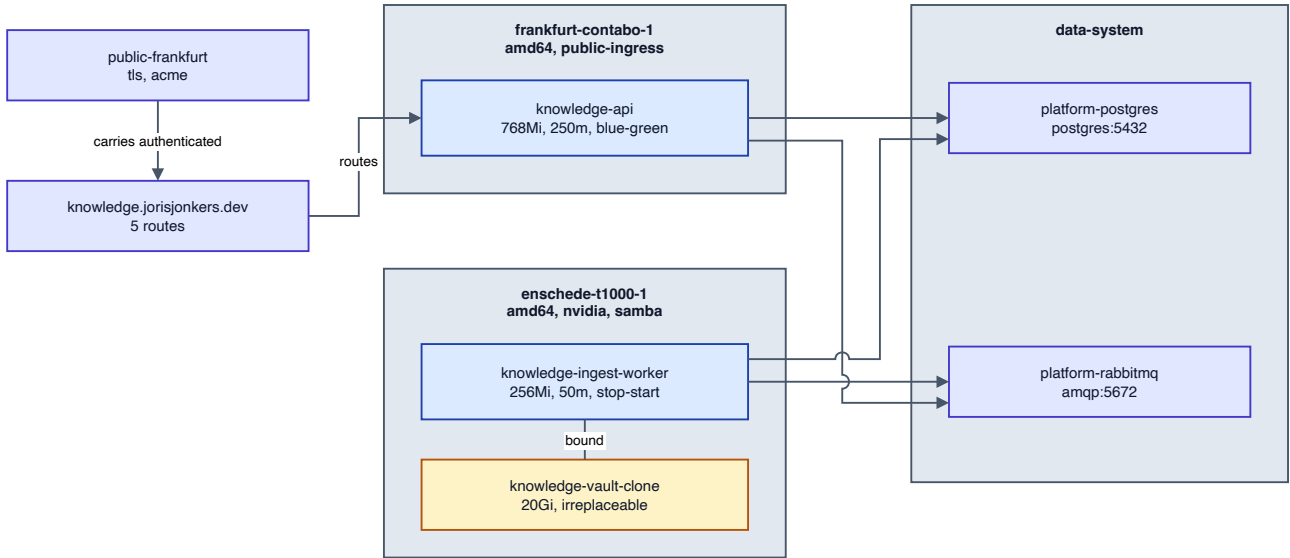


Figure 4: The knowledge Example's Resolved Model as a Deployment View

4.4 Evaluation Against the Purpose

Table 8 relates each observation of Section 1.5.1 to the part of the metamodels that addresses it, and to the models that show it.

Observation	How the metamodels address it	Shown by
Repeated values	A hostname is declared once, on an Exposure , and a port once, on a Surface . Routes and dependencies refer to them by name.	All examples; unknown-surface
Repeated derivations	The resolved model derives switchover, probe timings and deadline from the declared cutover and startup budget.	Table 9
Mechanisms instead of requirements	cutover , placement and writable paths state requirements. E_CUTOVER_UNHONOURABLE rejects a continuous process with a volume only one pod can mount.	cutover-continuous-over-rwo
Shared values	The platform document holds them, and constraints check a project document against it.	The 15 rejected pairs

Table 8: Observations, How the Metamodels Address Them, and the Models That Show It

4.5 Limitations

- The constraints embedded in the archive's **project-intent.ecore** do not see a second document. A rejected pair is reproduced in Eclipse only with both XMI files and **project-intent.ocl** loaded.
- Only the minimal example has a resolved model, written by hand. The resolved models of the other examples come from the Task 2 transformation, so the target metamodel is tested less than the source metamodel.
- Memory and CPU are checked per machine, not summed across the cluster (Section 2.2.3).
- Constraints over all project documents at once, such as unique application identifiers, are not in OCL.
- The concrete syntax covers only the YAML subset of the existing configuration.
- The platform example is parsed without the project documents that declare the applications it refers to, so six references in its XMI stay unresolved.
- Figure 1 leaves out two dependencies of **Placeholder**, to **Grant** and **Exposure**.

4.6 Reflection

Separating the authored and the resolved model kept both metamodels small. The source metamodel holds only what an author must write, and the target metamodel holds only the platform's decisions. Enumerations, and

naming each invariant after its code, made the rejected models easy to check, because each reports one known code. The constraints across documents were the hardest part: OCL evaluates one object at a time, so a rule between a platform document and a project file needs `allInstances()` and a guard, and Eclipse evaluates such rules only with the OCL document loaded. Without overrides, a wrong derivation rule affects every process of its kind, so the Task 2 transformation needs tests for each rule.

A Worked Examples

This appendix gives archive paths for three examples (Appendix A.4).

A.1 The Minimal Model, End to End

The input is `examples/minimal/notes.project.yml`. EMF does not write attributes that hold their default value, so `lifecycle` and `alertClass` are missing from `notes.project.xmi` beside it. Table 9 compares the authored values with those recorded in the hand-written resolved model, `minimal.resolveddeployment` (Section 4.2).

Authored	Derived in the resolved model
<code>project: notes</code>	<code>namespace="notes-system"</code> , and the reconcile unit (Section 3.2)
<code>cutover: continuous</code>	<code>switchover="blue-green"</code> , and a <code>releaseGate</code> with this process as its one member
<code>startupBudget: 20s</code>	a startup probe of 4 checks 5 seconds apart, and a deadline of 60 seconds
<code>runtime: node</code>	the checks <code>error-rate</code> and <code>latency</code> , because the profile exposes HTTP server metrics
<code>image: notes-api</code>	the image digest, and the numeric user and group it runs as
<code>audience: anonymous</code>	the edge tier <code>public-frankfurt</code> , and a middleware chain without an authentication check
<code>placement</code>	all seven machines are eligible, with no further placement restriction

Table 9: The Minimal Model: Authored and Derived Values

A.2 Continuous and Interrupted Applications in One Project

The input is `examples/knowledge/knowledge.project.yml`, with the two applications of Table 7. In the resolved model, the `continuous` API requires a release gate and its declared migration. The `interrupted` worker has neither: it declares no migration, and its old version stops before the new one starts (Section 3.2).

A.3 A Rejected Model

`refusals/prepare-process-serves.project.yml` declares a prepare process with a port, probes, replicas and a cutover. These belong to serving processes. Its XMI is `prepare-process-serves.project.xmi`. Validating that XMI reports `E_PREPARE_PROCESS_SERVES` on the second process of the first application.

A.4 The Hand-in Archive

The archive holds this report as `report.pdf`, a `README.md` with the steps below, and two folders. The first, `task-1-metamodelling/`, is an Eclipse project with OCL and Xtext natures, imported with File > Import > Existing Projects into Workspace. It contains the metamodels, five accepted examples and 33 rejected models: 18 single files and 15 document sets. `implementation/` contains the EMF implementation (Section 3.4) and the examples its tests read. Table 10 gives the layout. Each authored YAML file appears beside its parsed XMI, with any referenced `env/` files.

Each XMI declares its metamodel in `xsi:schemaLocation`, allowing Eclipse to open it without a registered package. The archive's `project-intent.ecore` embeds the OCL invariants and helpers. Plain Validate reports single-document codes. For cross-document sets, open the project XMI, add the platform XMI with Load Resource, then select OCL > Load Document and load `model/project-intent.ocl` before validating. Only `minimal` includes a resolved XMI. Task 2 will produce the others.

`./mvnw verify` in `implementation/emf` builds the implementation and runs its tests; it needs JDK 21. For each example and rejected model it writes the parsed model or the diagnostics, and the XMI, under `bundles/cli/target/parity/`. To open the implementation in Eclipse, import `implementation/emf` with File > Import > Existing Maven Projects after setting `emf.target` as the target platform.

Path	What it holds
<code>report.pdf</code>	this report
<code>task-1-metamodelling/model/</code>	<code>project-intent.ecore</code> with <code>project-intent.ocl</code> , the constraints of Section 3.3, and <code>resolved-deployment.ecore</code> , each metamodel with its <code>.genmodel</code>
<code>.../examples/<name>/</code> <code>.../examples/minimal/</code>	the authored <code>*.project.yml</code> , its <code>env/</code> files, and its XMI, <code>*.project.xmi</code> also <code>minimal.resolveddeployment</code> , the resolved model as an instance of the target metamodel
<code>.../examples/platform/</code> <code>.../refusals/</code>	the platform document, <code>platform.intent.yml</code> , and its XMI 18 single-file cases and 15 document sets, each with authored files and XMI. <code>README.md</code> gives codes and reproduction steps
<code>implementation/emf/</code> <code>implementation/spec/v1/examples/</code>	the Eclipse plug-in projects of Section 3.4, with the Maven build the authored models with their expected results, which the tests read

Table 10: Layout of the Hand-in Archive

B All Constraints

Table 11 lists each invariant of Section 3.3 by the code it reports, with its context class.

Code	Context	What it rejects
E_ALERT_CLASS_WITHOUT_SIGNAL E_CUTOVER_UNHONOURABLE	Observability Process	An alert level is set, but no metrics are collected. A continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_CUTOVER_MISSING	Process	A process with no cutover value at any level. A prepare process is exempt, because it replaces no running version.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	An application whose processes have different cutover values, although it switches as one unit.
E_PREPARE_PROCESS_SERVES	Process	A prepare process with a port, probes, replicas or a cutover , which only a serving process has.
E_SHARED_DECLARATION_DUPLICATED	seven classes	A shared declaration repeated unchanged at a lower level, where it has no effect.
E_SHARED_QUANTITY	Placement	A memory or CPU quantity declared above process level, where it would mean something different for each process.
E_PLACEMENT_INCOMPLETE	Process	A process with no memory or CPU quantity of its own. No higher level may supply one.
E_OWNER_ROLE_GRANTED	DatabaseGrant	A hand-written grant of the database role that changes the schema. The platform derives this role for the migration only.
E_MIGRATION_OWNER_DUPLICATED	Migration	A second application in one project that declares a migration for the project's database.
E_ENGINE_WITHOUT_DURABILITY	Process	A backup engine declared where no volume's durability class requires a backup.
E_DURABILITY_WITHOUT_ENGINE	Volume	A volume that requires a backup, with no engine to state the backup method.
E_ENV_CANNOT_RELOAD	Rotation	A secret that tolerates a reload but is delivered as an environment variable, which is read only at start.
E_ILLEGAL_DELIVERY_FOR_ACCESS	KvGrant	custody delivered as an environment variable or file, although its paths appear only at runtime.
E_NON_KV_DELIVERY	TransitGrant, DatabaseGrant	Also self-renew delivered as an environment variable, which its identity cannot read. A transit or database grant delivered as an environment variable or file. The application requests transit operations from the secret store and fetches database credentials issued per lease.
E_DUPLICATE_ROUTE_MATCH	Route	Two routes of one exposure with the same path and match type, which the derived precedence cannot order.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	A tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_HANDOVER_BOTH_PATHS	HandoverLedger	A project listed on both delivery paths, so two sources would both apply it.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	An audience no declared edge tier serves, so the route reaches no edge.
E_NO_ENGINE_POLICY	Process	An engine with no backup method in the platform document.
E_NO_DURABILITY_POLICY	Volume	A durability class with no policy in the platform document.
E_SECRETS_AT_REST_REQUIRED	Grant	A secret written into a cluster that does not encrypt secrets at rest.
E_MIGRATION_UNDECLARED	Application	An application that uses a database but declares no migration.
E_MIGRATION_WITHOUT_DATABASE	Migration	A migration on an application that uses no database, checked only when the documents of all its providers are loaded.
E_CREDENTIALS_WITHOUT_DATABASE	Credentials	A rotation override on a dependency whose provider has no database.
E_NO_MIGRATION_POLICY	Migration	A changelog, where the platform document defines no migration runner to build it on.
E_NO_DELIVERY_POLICY	Application	A continuous application, where the platform document defines no check interval for its blue/green switch.
E_HANDOVER_UNLISTED	Project	A project that the handover ledger lists on neither delivery path.

Table 11: All Constraints, Their Context Classes and What They Reject